

1. The AI-assisted workflow drill

Round-1-vague using cursor for prompt.

Round 1 branch:

<https://github.com/haidersara/flyrank-frontend-capstone/tree/round-1-vague>

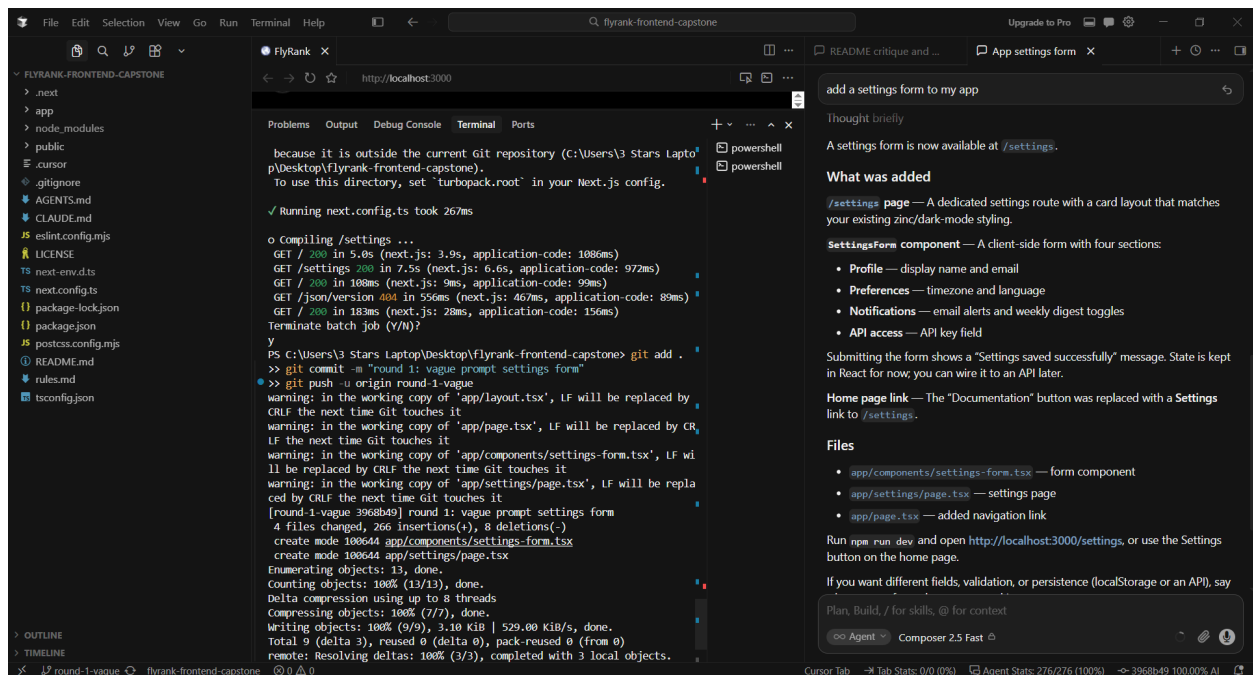
one lazy sentence, no context — "add a settings form to my app"

Result: AI invented its own scope — added Profile (name, email), Preferences (timezone, language), Notifications (email alerts, weekly digest), and API access (API key) — way beyond what was actually needed

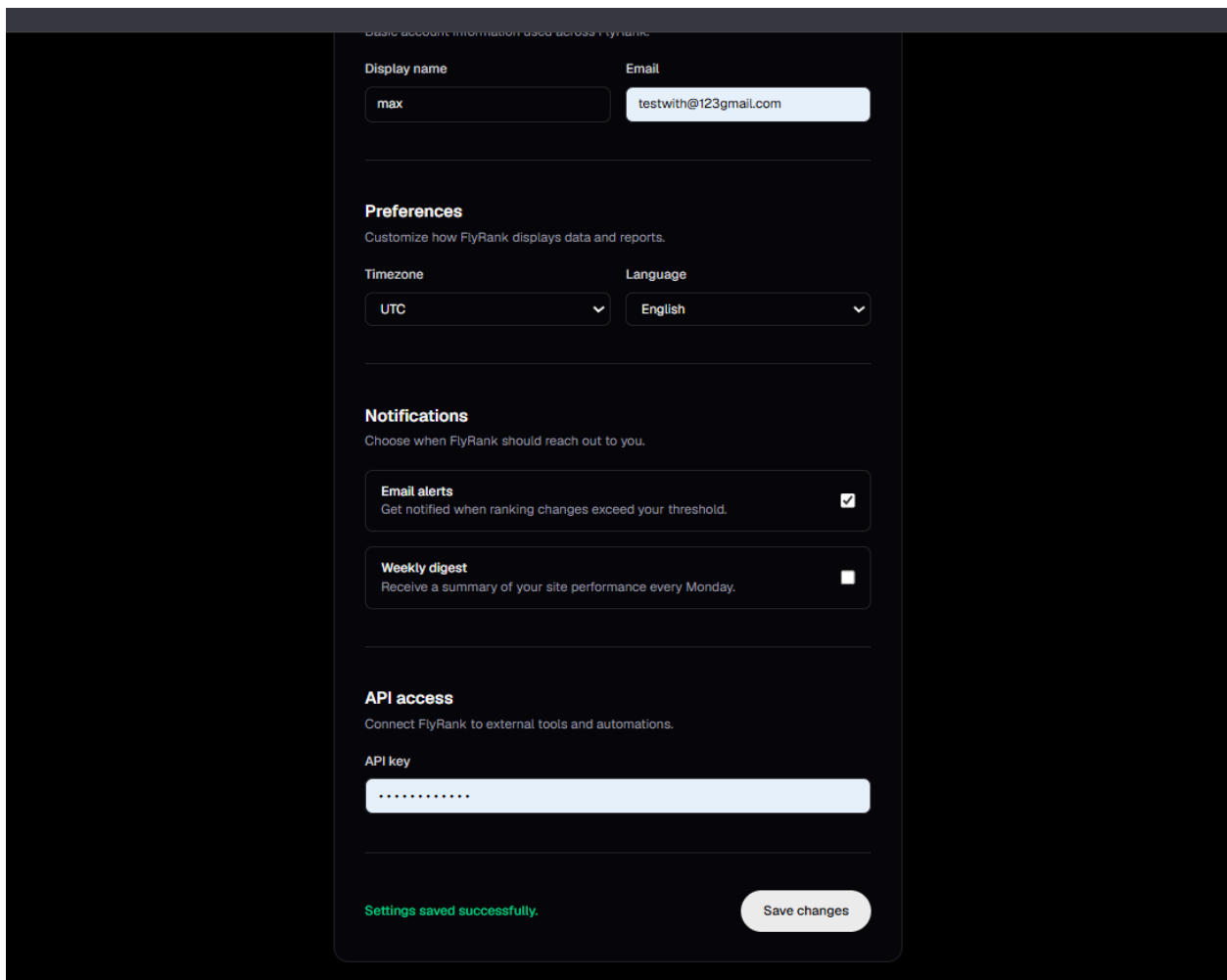
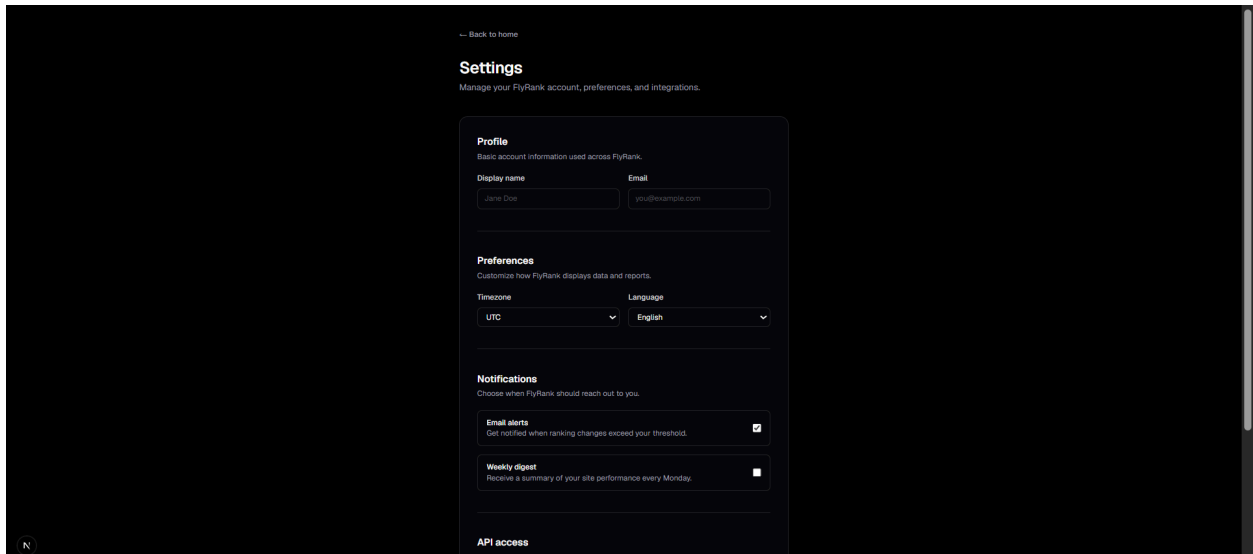
Gaps found through manual testing:

1. Email field only has passive browser-level `type="email"` validation — no custom app validation, no error styling
2. Display name has **zero validation** — empty name saves as "successful"
3. No accessibility checks were verified (labels, keyboard nav, error announcements)
4. "Settings saved successfully" is fake — just React state, nothing persisted

Committed and pushed to `round-1-vague`



Open <http://localhost:3000> in your browser:



Round 2: <https://github.com/haidersara/flyrank-frontend-capstone/tree/round-2-precise>

Round 2 (precise prompt):

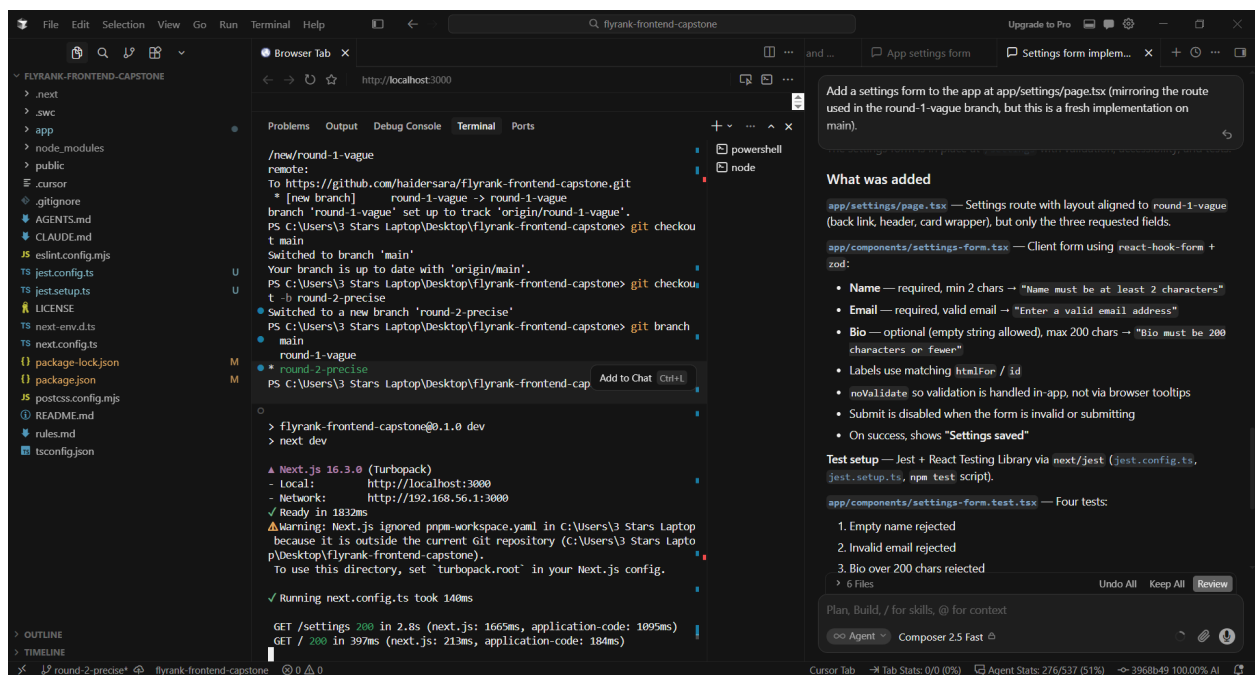
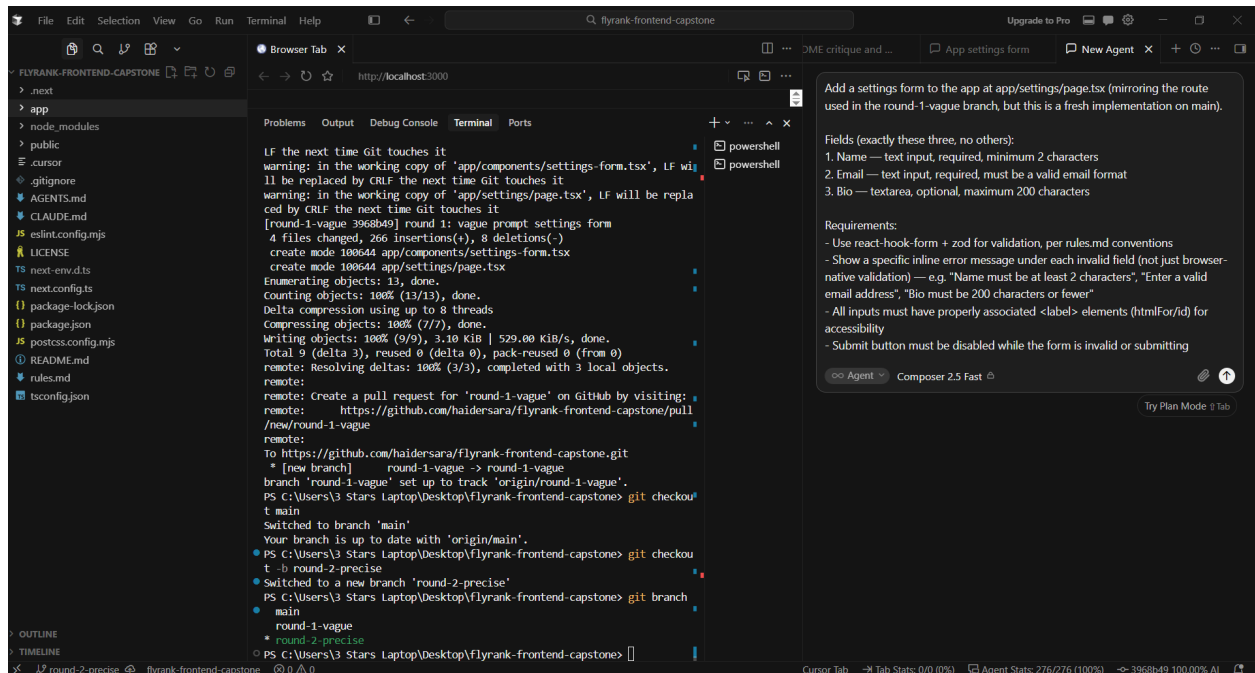
Same settings form, but this time the prompt specified exactly 3 fields (Name, Email, Bio), required react-hook-form + zod validation, mandated accessible labels, and demanded a verification step (write tests, run them). Result: real in-app validation with specific error messages (not browser tooltips), Save button disabled until valid, matched `htmlFor/id` labels, and 4 passing automated tests covering empty name, invalid email, over-length bio, and valid submission. Confirmed manually too — empty name and a bad email both got blocked with visible red error text, unlike round 1.

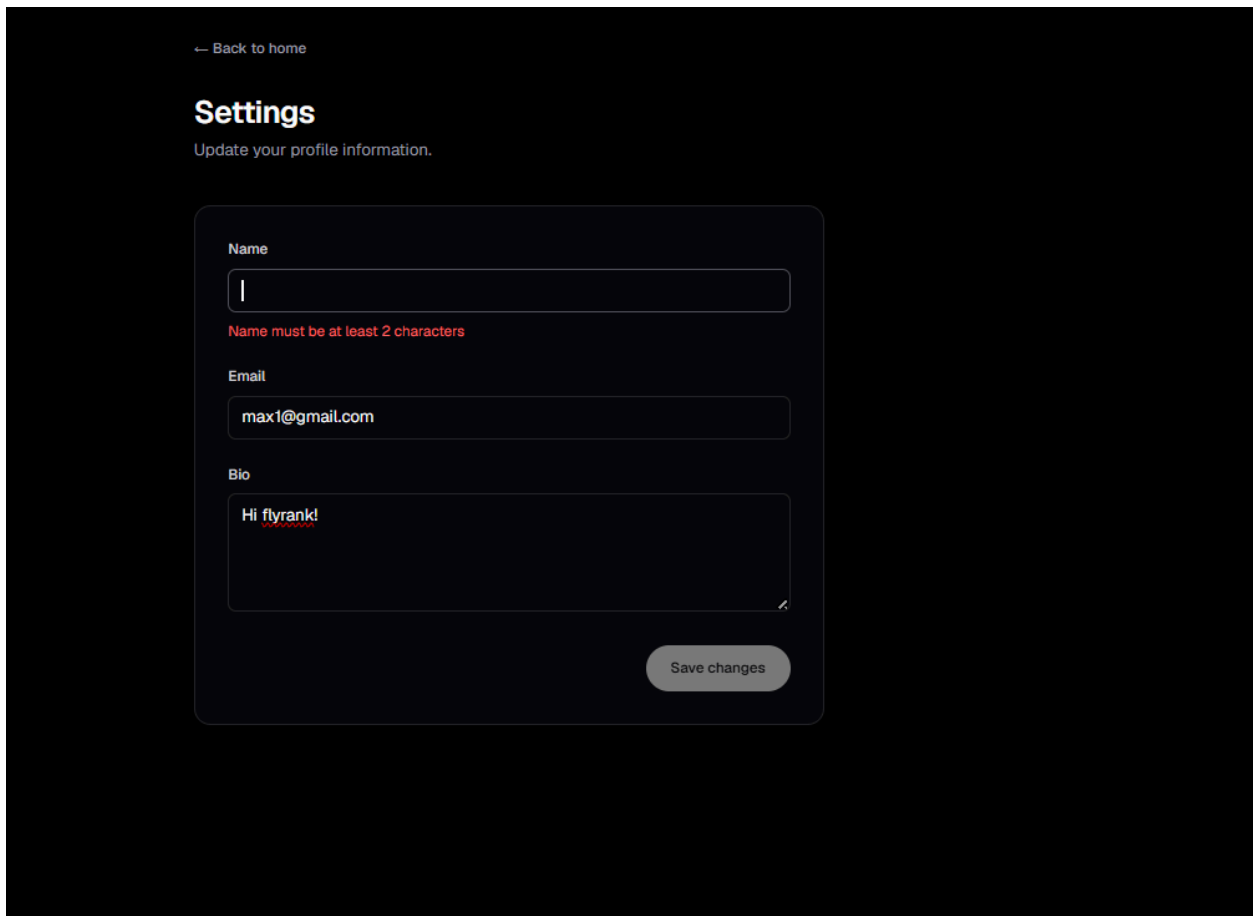
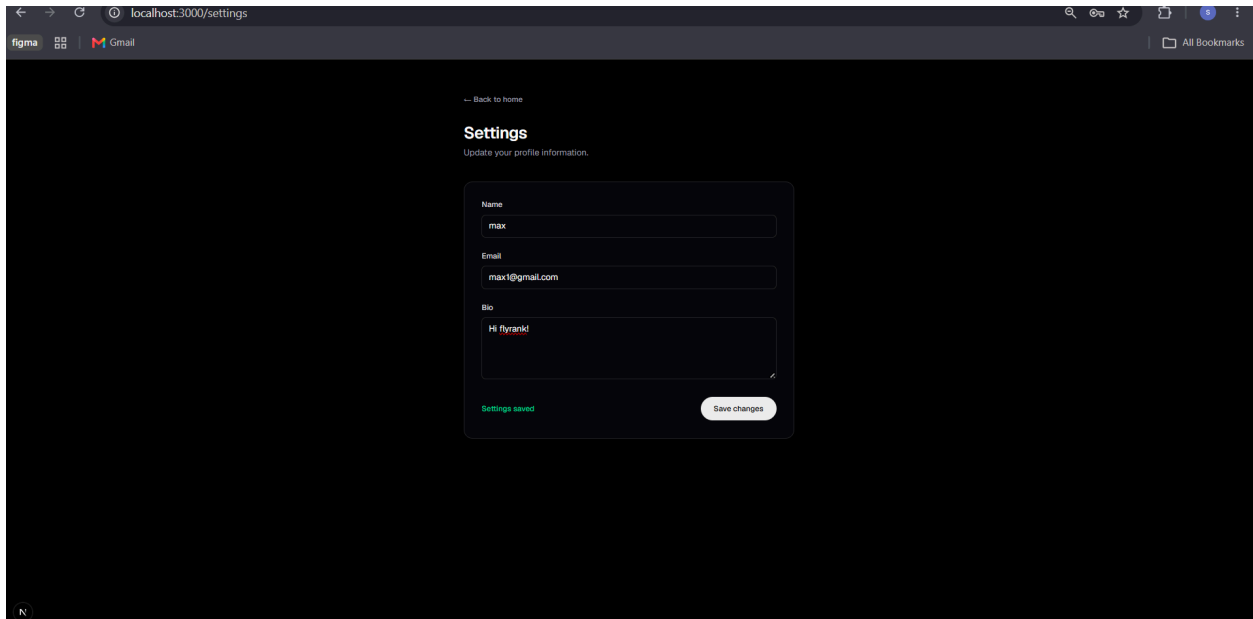
WORKFLOW.md (300–500 word writeup):

Compared both rounds across correctness, accessibility, edge cases, and review effort. Key findings:

- Round 1 had zero real validation (empty name saved fine) and quietly invented 4 extra fields (timezone, language, notifications, API key) nobody asked for — a genuine diff-backed scope-creep problem, not just weak validation
- Round 2 stayed exactly in scope (3 fields only) and enforced every rule with proper error messages
- Named the specific AI mistake caught: round 1's email field only worked because of the browser's native tooltip, not actual app logic — easy to miss, easy to break
- Noted that round 1 felt faster to prompt but slower to trust (had to manually poke every field to find gaps), while round 2 took longer to prompt but reviewed almost instantly since the tests already proved correctness

Ended with 3 concrete rules added to `rules.md`: always use react-hook-form + zod (never native HTML validation), always show inline app-rendered errors, and never let a vague prompt add unrequested fields.





[← Back to home](#)

Settings

Update your profile information.

Name

Email

Enter a valid email address

Bio

Save changes

Display name

Jane Doe

Email

test@gmail.com

Preferences

Customize how FlyRank displays data and reports.

Timezone

UTC

Language

English

Notifications

Choose when FlyRank should reach out to you.

Email alerts

Get notified when ranking changes exceed your threshold.



Weekly digest

Receive a summary of your site performance every Monday.



API access

Connect FlyRank to external tools and automations.

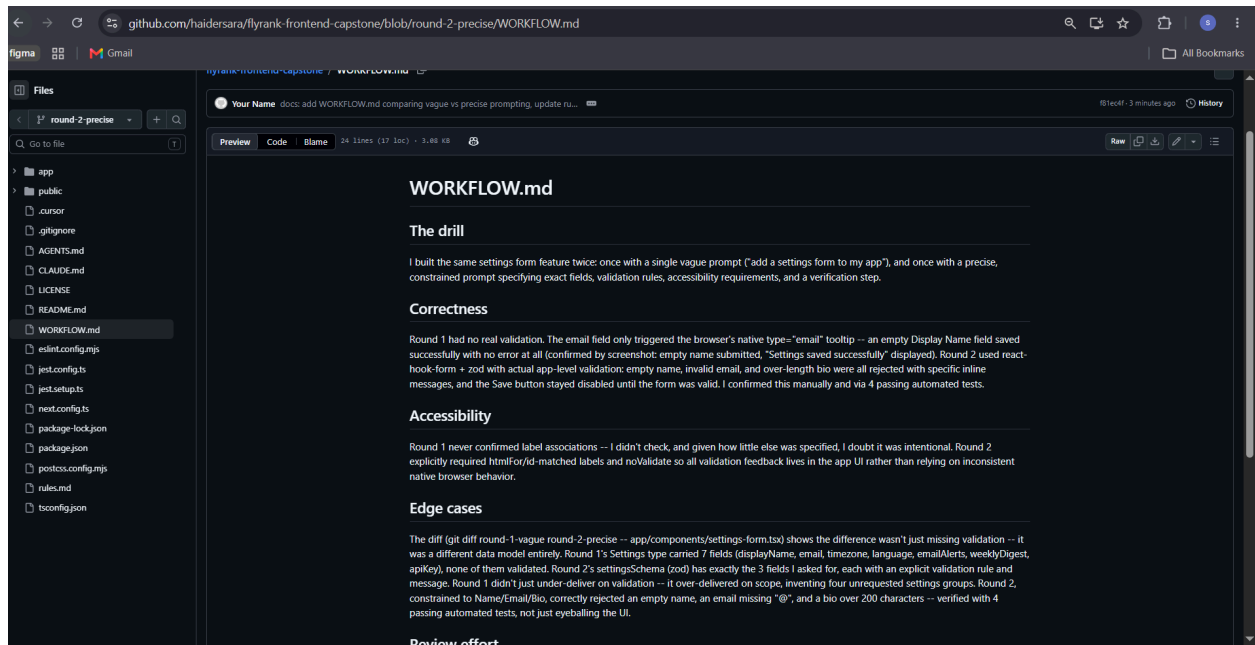
API key

.....

Settings saved successfully.

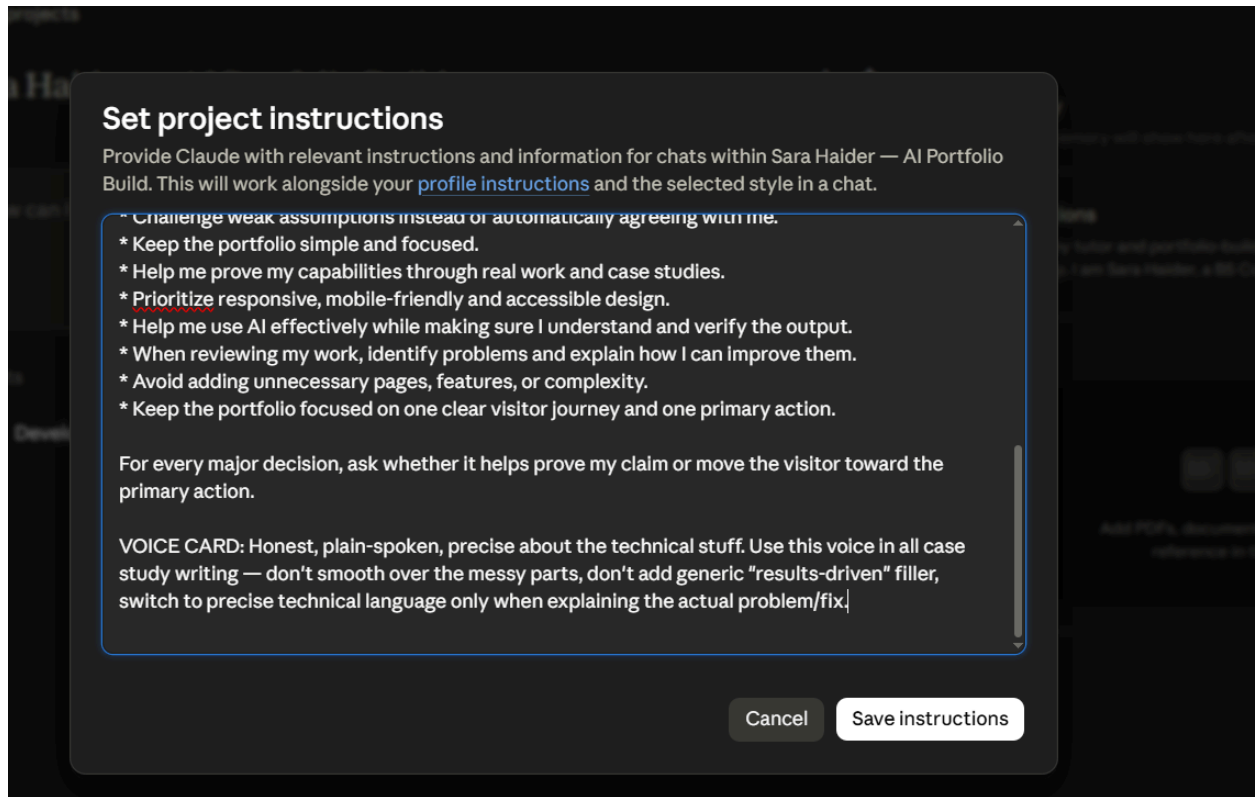
Save changes

<https://github.com/haidersara/flyrank-frontend-capstone/blob/round-2-precise/WORKFLOW.md>



2. Frame It as Cases: Work That Speaks for Itself

Locked in: **"Honest, plain-spoken, precise about the technical stuff"**



Everything inside the google doc:

<https://docs.google.com/document/d/1mgjMhqTMY2IFs9fHqe4H9SBrWQ04I498JudwC2Xhlm4/e/dit?usp=sharing>

Frame It as Cases — short summary: We built framing for all four sitemap case studies (CarryClicks, Melodies, AI-Powered Surveillance System, PBL Management System) by having you interview-answer one question at a time: the problem, what you actually did, and what came of it. You answered honestly and specifically instead of writing polished copy upfront, which is what made the case studies sound like you instead of generic AI output.

We locked in a voice card — *"Honest, plain-spoken, precise about the technical stuff"* — pulled directly from how you naturally explained your work (admitting what was confusing, getting excited about specific details, switching to precise terms only for the real technical problem). Each case study ended up with the required three beats, backed by concrete specifics you gave me: the CarryClicks foreign-key/payload bug, Melodies' background audio sync, the surveillance system's 5-10 second lag cut to under a second, and PBL's access-control isolation (honestly framed as partly still in progress).

We also wrote a bio and contact/CTA (download CV, or email), and closed with the required before/after: one generic AI line versus your actual edited line ("It went from a buggy class assignment to feeling like Spotify"), showing exactly what "your voice" means in practice.

3. The Prompt Ladder

<https://docs.google.com/document/d/1jc0Qsz8UOyTKONhhwQUbQaGWdPrZm1Xro9KMoVCarS0/edit?usp=sharing>

4. Prompting Fundamentals on Real Tasks v2

https://docs.google.com/document/d/1MeQ64HMYEFG4qwFtGvYRb_WhVJ6lFdh7v2xVLafUvrM/edit?usp=sharing